SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA1223

COURSE OUTCOME COVERED

CO1: Components of a computer system, including CPU, memory, and I/O. Basic Operational Concepts: Instruction execution cycle, instruction fetch and decode.Bus Structures: Single-bus, multi-bus, and hierarchical buses.Factors of Performance: CPU execution time, CPI, clock cycles, and benchmarking.8085 and 8086 Architectures: Overview, instruction sets, and addressing modes. Modern Architectures: ARM Cortex, Intel Core, and AMD Ryzen.Instruction Sets, Formats, and Addressing Modes. Number Systems: Binary, hexadecimal, and their applications in computer systems. (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25** 100

62.5

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Adarsh Reddy V (192525412) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V Adarsh Reddy
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode– execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	3	3	3	2	14
Q2	3	3	3	3	2	14
Q3	2	2	2	2	1.5	9.5
Q4	3	2	2	3	1.5	11.5
Q5	2	3	3	3	2	13

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

62

Understan ding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements 4	Good understanding with minor gaps 3	Basic understanding, some requirements unclear 2	Poor understanding or misinterpretation of the problem 1
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems 6	Applies relevant concepts correctly with minor errors 5	Applies basic concepts with limited accuracy 4	Fails to apply appropriate concepts 1-2
ProblemSolving Approach	20	Logical, wellstructured, and efficient approach 4	Mostly logical approach 3	Basic approach with some inconsistencies 2	Illogical or unclear approach 1
			with minor gaps		
Accuracy of Solution	20	Completely correct solution with proper justification 4	Mostly correct with minor errors 3	Partially correct solution 2	Incorrect or incomplete solution 1
Clarity	10	Clear, wellorganized, and properly explained solution 2	Understandable with minor clarity issues 1.5	Limited clarity and explanation 1	Poorly presented and difficult to understand 0

ANSWERS

Question-1:

A smart city surveillance system continuously receives video data from multiple cameras, data flows from **I/O devices (cameras)** to **memory**, where the **CPU** fetches and process for threat detection. During peak hours, the large amount of data causes delays because CPU, memory, and I/O devices compete for system resources.

1. Interaction between CPU, Memory, and I/O

- **I/O Devices:** Cameras continuously send high-resolution video streams.
- **Memory (RAM):** Temporarily stores video frames before processing.
- **CPU:** Fetches video data from memory, processes it, and detects threats.

When many cameras send data simultaneously:

- Memory becomes overloaded with incoming data.
- The CPU spends more time waiting for data from memory.
- I/O devices wait for bus access to transfer data.
- This creates **bus congestion**, increasing processing delay.

2. Effect of Bus Structure

a) Single-Bus Structure

- CPU, memory, and I/O devices share one common bus.
- Only one data transfer can occur at a time.
- Heavy traffic causes contention and delays.
- Overall system performance decreases.

Advantages:

- Simple design
- Low cost **Disadvantages:**
- Bus bottleneck
- Slow data transfer during high load
- Increased CPU waiting time

b) Multi-Bus Structure

- Separate buses are used for CPU-memory and I/O communication.
- Multiple data transfers occur simultaneously.
- Reduces bus congestion.
- Improves throughput and response time.

Advantages:

- Faster communication
- Better parallel processing
- Reduced waiting time
- Suitable for real-time surveillance systems

3. Improving the Instruction Execution Cycle

The CPU follows the **Fetch → Decode → Execute** cycle.

Performance can be improved using:

- **Pipelining:** Executes multiple instructions simultaneously.

- **Cache Memory:** Stores frequently used data, reducing memory access time.
- **DMA (Direct Memory Access):** Allows I/O devices to transfer data directly to memory without CPU involvement.
- **Branch Prediction:** Reduces delays caused by conditional instructions.
- **Multi-core Processors:** Different cores process multiple video streams simultaneously.

These techniques reduce execution time and improve system efficiency.

4. Performance Enhancement

By replacing a **single-bus** system with a **multi-bus** architecture and improving instruction execution through pipelining, cache memory, DMA, and multi-core processing:

- Data transfer becomes faster.
- CPU waiting time decreases.
- Bus congestion is minimized.
- Real-time threat detection improves.
- Overall surveillance system performance becomes faster and more reliable.

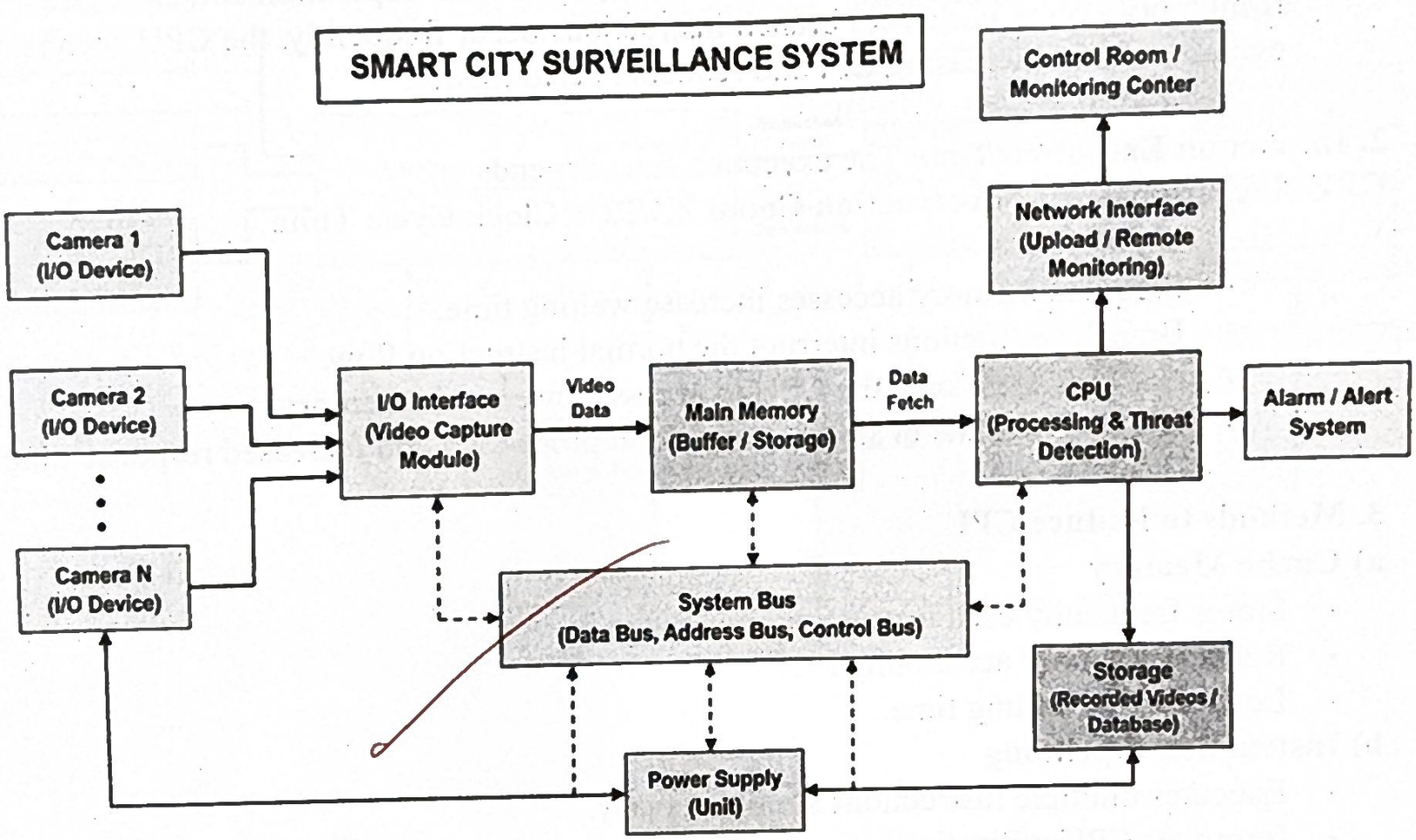


Fig. No 7

Conclusion :

The smart city surveillance system processes continuous video data from multiple cameras through I/O devices, memory, and the CPU. During peak hours, heavy data traffic causes resource contention among the CPU, memory, and I/O devices, resulting in processing delays. By improving memory management, increasing bus bandwidth, using buffering, DMA, and

efficient CPU scheduling, the system can reduce delays and process video data more efficient
This ensures faster threat detection, better system performance, and reliable real-ti
surveillance.

Question-2:

A real-time stock trading application must process transactions quickly with minimum dela
During high load, the CPU experiences a **high CPI (Cycles Per Instruction)** due to freque
memory access and branch instructions, which increases execution time and delays transactio
processing.

1. Fetch–Decode–Execute Cycle
The CPU executes every instruction in three main stages:

- **Fetch:** The CPU fetches the instruction from memory.
- **Decode:** The control unit decodes the instruction and determines the required operation.
- **Execute:** The ALU or execution unit performs the operation and stores the result.

If memory access is slow or branch instructions occur frequently, the CPU spends more cycles waiting, increasing CPI and reducing performance.

2. Impact on Execution Time The execution time depends on:
CPU Execution Time = Instruction Count × CPI × Clock Cycle Time
In this scenario:

- Frequent memory accesses increase waiting time.
- Branch instructions interrupt the normal instruction flow.
- Cache misses force the CPU to access slower main memory.
- High CPI results in slower transaction processing and increased response time.

3. Methods to Reduce CPI
a) Cache Memory

- Stores frequently used instructions and data.
- Reduces memory access time.
- Lowers CPU waiting time.

b) Instruction Pipelining

- Executes multiple instructions simultaneously.
- Improves CPU utilization.
- Increases instruction throughput.

c) Branch Prediction

- Predicts the outcome of branch instructions.
- Reduces pipeline stalls.
- Improves execution speed.

d) Superscalar Architecture

- Executes multiple instructions in one clock cycle.
- Increases instruction-level parallelism.

- Reduces overall CPI.

e) Multi-Core Processors

- Different processor cores execute different transactions simultaneously.
- Improves performance during heavy workloads.

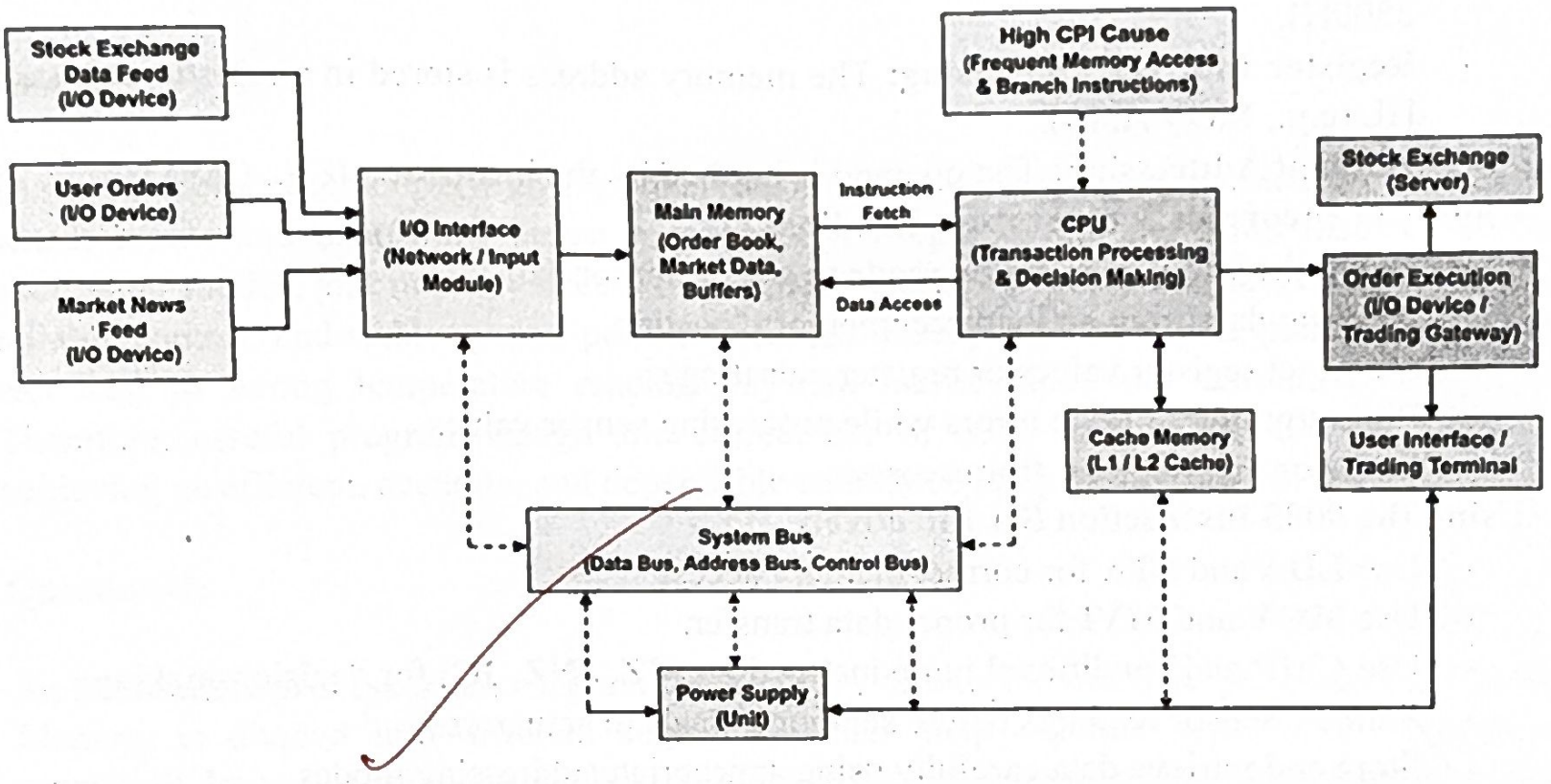
f) Prefetching

- Loads instructions and data into cache before they are needed.
- Reduces memory access delays.

4. Performance Improvement

Using the above architectural enhancements:

- CPI decreases.
- Memory access becomes faster.
- Branch delays are minimized.
- More instructions are executed per second.
- Transaction processing becomes faster and more reliable.



Conclusion:

The performance of a real-time stock trading application is highly dependent on CPU efficiency. A high Cycles Per Instruction (CPI) caused by frequent memory accesses and branch instructions increases execution time, leading to delays in transaction processing. By reducing memory latency, improving cache performance, optimizing branch prediction, and lowering CPI, the CPU can execute instructions faster. This improves throughput, reduces transaction

delays, and ensures reliable and efficient real-time stock trading, even during periods of high system load.

Question-3:

An embedded system based on the **8085 microprocessor** is used in industrial temperature control, where sensors provide input and actuators respond accordingly. Incorrect temperature readings may occur due to improper use of **addressing modes**, incorrect memory access, or programming errors. Proper use of the 8085 instruction set ensures accurate and reliable system operation.

Addressing Modes in 8085:

The 8085 supports different addressing modes for accessing data:

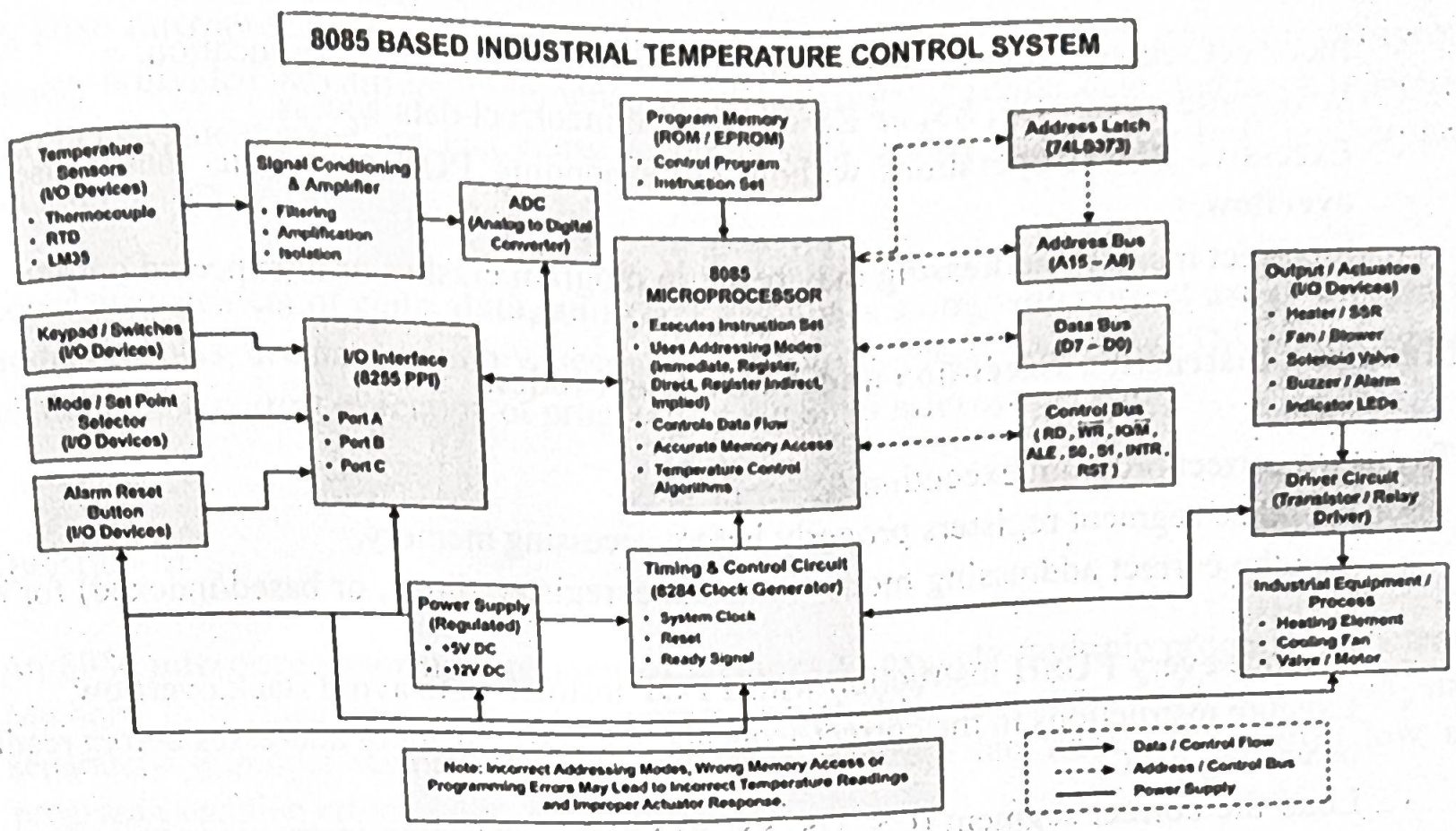
- **Immediate Addressing:** Data is given directly in the instruction (e.g., MVI A, 25H).
- **Register Addressing:** Data is stored in CPU registers (e.g., MOV A, B).
- **Direct Addressing:** The memory address is specified in the instruction (e.g., LDA 2500H).
- **Register Indirect Addressing:** The memory address is stored in a register pair such as HL (e.g., MOV A, M).
- **Implicit Addressing:** The operand is implied by the instruction (e.g., CMA).

Causes of Incorrect Temperature Readings

- Using the wrong addressing mode to access sensor data.
- Reading data from an incorrect memory location.
- Incorrect register values or register pair usage.
- Programming or logic errors while processing sensor values.
- Noise or invalid input from sensors.

Using the 8085 Instruction Set Effectively

- Use **LDA** and **STA** for correct memory access.
- Use **MOV** and **MVI** for proper data transfer.
- Use **CMP** and conditional jump instructions (**JZ, JNZ, JC**) for decision-making.
- Validate sensor readings before sending signals to actuators.
- Store and retrieve data carefully using appropriate addressing modes.



Conclusion :

The 8085 microprocessor plays an important role in industrial temperature control by processing sensor inputs and controlling actuators accurately. Proper use of the 8085 instruction set, addressing modes, and memory access techniques ensures correct temperature measurement, reliable control, and stable system performance. Incorrect programming or memory handling can lead to wrong temperature readings, system malfunction, and unreliable operation. Therefore, careful program design and correct use of 8085 instructions are essential for achieving an efficient, accurate, and dependable embedded temperature control system

Question-4:

An **8086 microprocessor** uses **segmented memory** to execute multiple programs efficiently. Memory is divided into different segments, which help organize code, data, and stack separately. Improper segment handling can cause incorrect data access, stack overflow, and program execution errors.

Segmentation in 8086:

The 8086 divides memory into the following segments:

- **Code Segment (CS):** Stores program instructions.
- **Data Segment (DS):** Stores variables and data.
- **Stack Segment (SS):** Stores temporary data, return addresses, and function calls.
- **Extra Segment (ES):** Used as an additional data segment.

Each segment uses a **segment register** and an **offset address** to access memory.

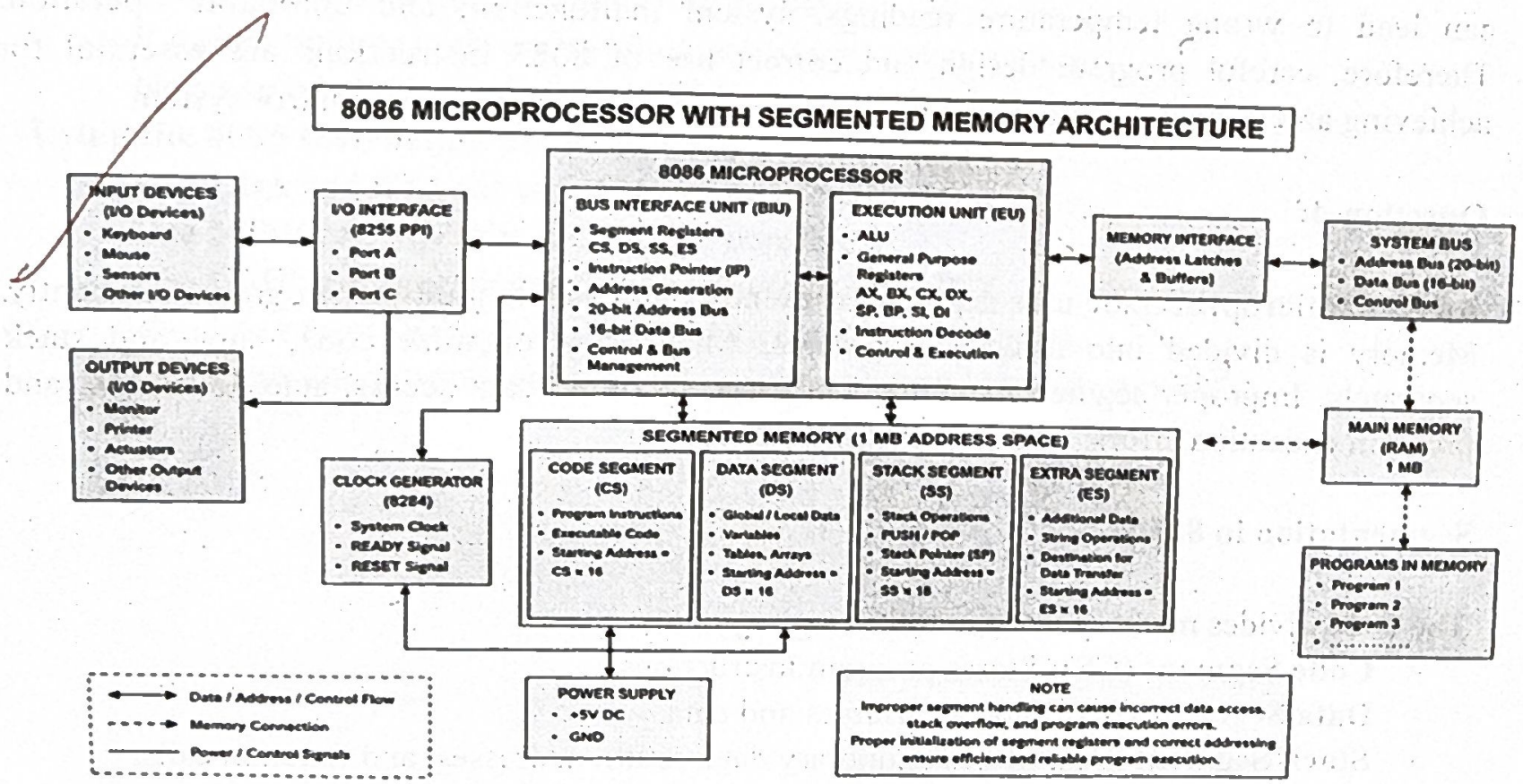
Problems Due to Improper Segment Handling:

- Incorrect segment register values may access the wrong memory location.
- Wrong use of **CS, DS, SS, or ES** can lead to incorrect data access.
- Excessive **PUSH** operations without corresponding **POP** operations may cause stack **overflow**.
- Incorrect memory addressing may result in program crashes or unexpected outputs.

Managing Instruction Execution and Addressing Modes:

To ensure correct program execution:

- Initialize segment registers properly before accessing memory.
- Use the correct addressing mode (immediate, register, direct, or based/indexed) for data access.
- Balance every **PUSH** instruction with a **POP** instruction to avoid stack overflow.
- Execute instructions in the correct sequence and verify memory addresses before reading or writing data.
- Load the correct segment (CS, DS, SS, ES) before executing instructions to ensure the processor accesses the intended memory area.
- Avoid accessing memory outside the segment limits to prevent invalid data access and program execution errors.
- Test and debug the program regularly to detect addressing errors, incorrect memory references, and stack-related problems before deployment.



Conclusion:

An 8086 microprocessor uses segmented memory to execute multiple programs efficiently. Memory is divided into different segments, which help organize code, data, and stack separately. Improper segment handling can cause incorrect data access, stack overflow, and program execution errors.

Proper management of **code, data, and stack segments** along with correct use of **addressing modes** ensures accurate memory access and prevents stack overflow. This improves the reliability and correct execution of programs in the **8086 microprocessor**.

Question-5:

An **8086 microprocessor** uses **segmented memory** to execute multiple programs efficiently. Memory is divided into different segments, which help organize code, data, and stack separately. Improper segment handling can cause incorrect data access, stack overflow, and program execution errors.

Segmentation in 8086:

The 8086 divides memory into the following segments:

- **Code Segment (CS):** Stores program instructions.
- **Data Segment (DS):** Stores variables and data.
- **Stack Segment (SS):** Stores temporary data, return addresses, and function calls.
- **Extra Segment (ES):** Used as an additional data segment.

Each segment uses a **segment register** and an **offset address** to access memory.

Problems Due to Improper Segment Handling:

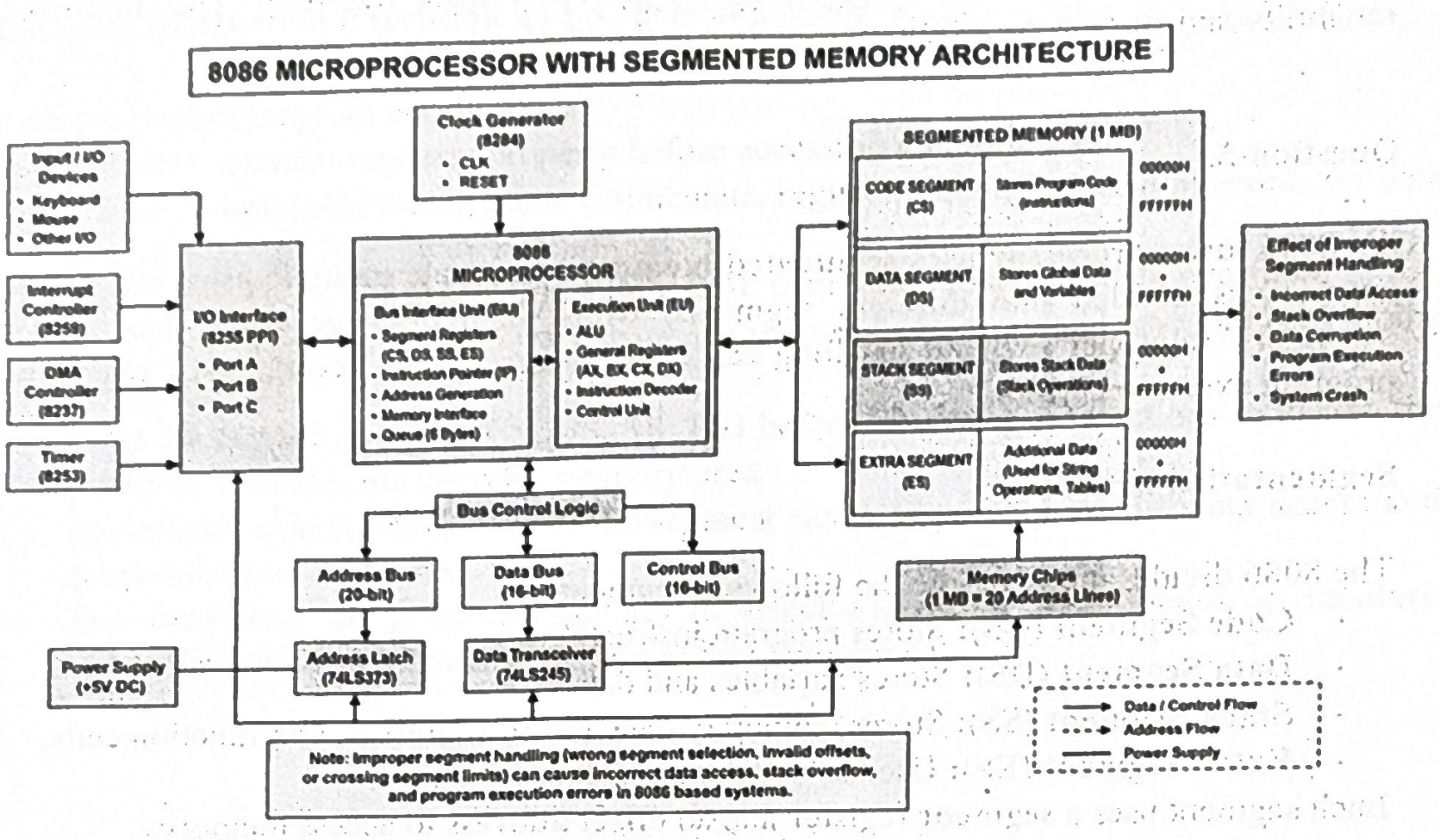
- Incorrect segment register values may access the wrong memory location.
- Wrong use of **CS, DS, SS, or ES** can lead to incorrect data access.
- Excessive **PUSH** operations without corresponding **POP** operations may cause **stack overflow**.
- Incorrect memory addressing may result in program crashes or unexpected outputs.

Managing Instruction Execution and Addressing Modes:

To ensure correct program execution:

- Initialize segment registers properly before accessing memory.
- Use the correct addressing mode (immediate, register, direct, or based/indexed) for data access.
- Balance every **PUSH** instruction with a **POP** instruction to avoid stack overflow.
- Execute instructions in the correct sequence and verify memory addresses before reading or writing data.

- Test and debug the program thoroughly to identify and correct logical and programming errors before deployment.
- Validate sensor input data before processing to prevent incorrect readings caused by invalid or noisy signals.
- Use proper interrupt handling and timing to ensure real-time response and reliable operation of the temperature control system.



Conclusion:

Proper management of the **code, data, and stack segments** is essential for the efficient operation of the **8086 microprocessor**. The **code segment** stores program instructions, the **data segment** holds variables and constants, and the **stack segment** is used for temporary data storage, function calls, interrupts, and return addresses. Correct organization and maintenance of these memory segments ensure that the processor accesses the intended instructions and data without conflicts or errors.

In addition, the proper use of **addressing modes** enables the processor to access operands accurately and efficiently from memory or registers. Choosing the appropriate addressing mode improves execution speed, optimizes memory usage, and simplifies program design. Effective stack management prevents issues such as **stack overflow** and **stack underflow**, which can lead to program crashes, data corruption, or unexpected behavior.